

Travel Booking API Documentation

Frontend Developer Reference

Backend Team

November 27, 2025

Contents

1	Introduction	2
2	Endpoints Overview	2
2.1	GET /bookings	2
2.2	POST /bookings	3
2.3	GET /bookings/explore	3
2.4	GET /profile/:id	4
2.5	POST /reviews	5
2.6	GET /reviews/:tourId	5
2.7	GET /agencies	6
2.8	GET /guides	6
2.9	GET /agencies/browse	7
2.10	GET /tours/browse	8
3	Frontend Integration Examples	9
3.1	Fetch with JavaScript	9
3.2	Axios Examples	10
4	Error Handling	11
4.1	HTTP Status Codes	11
4.2	Common Error Messages	11
5	Notes	11

1 Introduction

This document provides complete API specifications for the Travel Booking System. All endpoints return JSON responses with a consistent structure:

```
{
  "success": true/false,
  "data": {...}, // on success
  "error": "..." // on error
}
```

2 Endpoints Overview

2.1 GET /bookings

Purpose: Retrieve bookings filtered by agency, guide, traveller name, or status.

Request Parameters (Query String)

- `agencyId` (string, **required if guideId not provided**)
- `guideId` (string, **required if agencyId not provided**)
- `travellerName` (string, optional) - Case-insensitive partial match
- `status` (string, optional) - e.g., "PENDING", "CONFIRMED"

Example Request:

```
GET /bookings?agencyId=agency_123&status=PENDING
```

Example Response (200 OK):

```
{
  "success": true,
  "data": [
    {
      "booking_id": "booking_1",
      "traveller_id": "traveller_1",
      "tour_id": "tour_1",
      "booking_date": "2025-01-15",
      "status": "PENDING",
      "travellers": {
        "traveller_fn": "John",
        "traveller_ls": "Doe"
      },
      "tours": {
        "tour_id": "tour_1",
        "tour_title": "Paris Adventure",
        "location": "Paris, France",
        "price": 1500.00,
        "agency_id": "agency_123",
        "guide_id": null,
        "start_date": "2025-02-01"
      }
    }
  ]
}
```

Error Response (400 Bad Request):

```
{
  "success": false,
  "error": "Either agencyId or guideId is required"
}
```

2.2 POST /bookings

Purpose: Create a new booking for a traveller.

Request Body (JSON)

- traveller_id (string, **required**)
- tour_id (string, **required**)

Note: booking_date defaults to today, status defaults to "PENDING". Booking is rejected if tour start date is in the past. **Example Request:**

```
POST /bookings
Content-Type: application/json
{
  "traveller_id": "traveller_1",
  "tour_id": "tour_1"
}
```

Example Response (201 Created):

```
{
  "success": true,
  "data": {
    "booking_id": "booking_456",
    "traveller_id": "traveller_1",
    "tour_id": "tour_1",
    "booking_date": "2025-11-27",
    "status": "PENDING"
  }
}
```

2.3 GET /bookings/explore

Purpose: Get all bookings for a specific user (traveller).

Request Parameters (Query String)

- userId (string, **required**)

Example Request:

```
GET /bookings/explore?userId=traveller_1
```

Example Response (200 OK):

```
{
  "success": true,
  "data": [
    {
```

```

    "booking_id": "booking_456",
    "traveller_id": "traveller_1",
    "tour_id": "tour_1",
    "booking_date": "2025-11-27",
    "status": "PENDING",
    "tours": {
      "tour_id": "tour_1",
      "tour_title": "Paris Adventure",
      "location": "Paris, France",
      "price": 1500.00,
      "agency_id": "agency_123",
      "guide_id": null,
      "start_date": "2025-02-01"
    }
  }
]
}

```

2.4 GET /profile/:id

Purpose: Retrieve profile details for an agency or guide.

Request Parameters

- id (URL parameter, **required**) - Agency or Guide ID
- type (query string, **required**) - Must be "agency" or "guide"

Example Request:

```
GET /profile/agency_123?type=agency
```

Example Response - Agency (200 OK):

```

{
  "success": true,
  "data": {
    "agency_id": "agency_123",
    "agency_name": "Grand Tours Inc",
    "rating": 450,
    "num_raters": 100,
    "manager_id": "user_1",
    "manager": {
      "email": "manager@grandtours.com",
      "role": "MANAGER"
    }
  }
}

```

Example Response - Guide (200 OK):

```

{
  "success": true,
  "data": {
    "guide_id": "guide_456",
    "guide_name": "Maria Garcia",
    "ratings": 380,
    "num_raters": 80,
    "user": {

```

```

    "email": "maria@example.com",
    "role": "GUIDE"
  }
}

```

2.5 POST /reviews

Purpose: Add a review for a tour and update agency/guide ratings.

Request Body (JSON)

- `tour_id` (string, **required**)
- `traveller_id` (string, **required**)
- `review_score` (number, **required**) - Rating value
- `comment` (string, optional) - Review text

Example Request:

```

POST /reviews
Content-Type: application/json
{
  "tour_id": "tour_1",
  "traveller_id": "traveller_1",
  "review_score": 5,
  "comment": "Amazing experience!"
}

```

Example Response (201 Created):

```

{
  "success": true,
  "data": {
    "review_id": "review_789",
    "tour_id": "tour_1",
    "traveller_id": "traveller_1",
    "review_score": 5,
    "comment": "Amazing experience!",
    "created_at": "2025-11-27T10:30:00Z"
  }
}

```

2.6 GET /reviews/:tourId

Purpose: Get all reviews for a specific tour.

Request Parameters

- `tourId` (URL parameter, **required**)

Example Request:

```

GET /reviews/tour_1

```

Example Response (200 OK):

```
{
  "success": true,
  "data": [
    {
      "review_id": "review_789",
      "tour_id": "tour_1",
      "traveller_id": "traveller_1",
      "review_score": 5,
      "comment": "Amazing experience!",
      "created_at": "2025-11-27T10:30:00Z",
      "travellers": {
        "traveller_fn": "John",
        "traveller_ls": "Doe"
      }
    }
  ]
}
```

2.7 GET /agencies

Purpose: Search agencies by name.

Request Parameters (Query String)

- **search** (string, **required**) - Search term (case-insensitive)
- **limit** (number, optional) - Default: 10, Max results to return

Example Request:

```
GET /agencies?search=grand&limit=5
```

Example Response (200 OK):

```
{
  "success": true,
  "data": [
    {
      "agency_id": "agency_123",
      "agency_name": "Grand Tours Inc",
      "rating": 450,
      "num_raters": 100,
      "manager_id": "user_1"
    }
  ]
}
```

2.8 GET /guides

Purpose: Search guides by name.

Request Parameters (Query String)

- **search** (string, **required**) - Search term (case-insensitive)
- **limit** (number, optional) - Default: 10, Max results to return

Example Request:

```
GET /guides?search=maria&limit=5
```

Example Response (200 OK):

```
{
  "success": true,
  "data": [
    {
      "guide_id": "guide_456",
      "guide_name": "Maria Garcia",
      "ratings": 380,
      "num_raters": 80
    }
  ]
}
```

2.9 GET /agencies/browse

Purpose: Browse all agencies with pagination, including their tours.

Request Parameters (Query String)

- **page** (number, optional) - Default: 1, must be ≥ 1
- **size** (number, optional) - Default: 10, range: 1-100

Example Request:

```
GET /agencies/browse?page=1&size=10
```

Example Response (200 OK):

```
{
  "success": true,
  "data": {
    "agencies": [
      {
        "agency_id": "agency_123",
        "agency_name": "Grand Tours Inc",
        "rating": 450,
        "num_raters": 100,
        "manager_id": "user_1",
        "tours": [
          {
            "tour_id": "tour_1",
            "tour_title": "Paris Adventure",
            "location": "Paris, France",
            "price": 1500.00,
            "start_date": "2025-02-01",
            "guide_id": null
          }
        ]
      }
    ],
    "pagination": {
      "page": 1,
      "size": 10,

```

```

    "total": 45,
    "totalPages": 5,
    "hasNext": true,
    "hasPrev": false
  }
}
}

```

2.10 GET /tours/browse

Purpose: Browse all tours with pagination, including agency and guide details. **Updated:** Now supports optional filtering by categories, regions, price range, and provider type.

Request Parameters (Query String)

- **page** (number, optional) - Default: 1, must be ≥ 1
- **size** (number, optional) - Default: 10, range: 1-100
- **cat** (string, optional) - Comma-separated list of categories (e.g., "adventure,hiking") for filtering by tour category
- **regions** (string, optional) - Comma-separated list of regions/locations (e.g., "Paris,France,Tokyo") for filtering by tour location
- **priceMin** (number, optional) - Minimum price filter (non-negative; null means no lower limit)
- **priceMax** (number, optional) - Maximum price filter (non-negative; null means no upper limit)
- **provider** (string, optional) - Filter by "agency" or "guide" (tours must have either agency_id or guide_id, not both)

Example Request:

```
GET /tours/browse?page=2&size=5&cat=adventure,hiking&regions=Paris,
France&priceMin=1000&priceMax=2000&provider=agency
```

Example Response (200 OK):

```

{
  "success": true,
  "data": {
    "tours": [
      {
        "tour_id": "tour_1",
        "tour_title": "Paris Adventure",
        "location": "Paris, France",
        "price": 1500.00,
        "start_date": "2025-02-01",
        "created_at": "2025-01-01T00:00:00Z",
        "agency_id": "agency_123",
        "guide_id": null,
        "category": "adventure",
        "agencies": {
          "agency_id": "agency_123",
          "agency_name": "Grand Tours Inc",

```

```

        "rating": 450,
        "num_raters": 100
      },
      "guides": null
    }
  ],
  "pagination": {
    "page": 2,
    "size": 5,
    "total": 120,
    "totalPages": 24,
    "hasNext": true,
    "hasPrev": true
  }
}

```

3 Frontend Integration Examples

3.1 Fetch with JavaScript

```

// Example: Get bookings for an agency
async function fetchAgencyBookings(agencyId) {
  try {
    const response = await fetch(
      `/bookings?agencyId=${agencyId}&status=PENDING`
    );
    const result = await response.json();

    if (result.success) {
      console.log('Bookings:', result.data);
      return result.data;
    } else {
      console.error('Error:', result.error);
    }
  } catch (error) {
    console.error('Network error:', error);
  }
}

// Example: Create a booking
async function createBooking(travellerId, tourId) {
  try {
    const response = await fetch('/bookings', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        traveller_id: travellerId,
        tour_id: tourId
      })
    });

    const result = await response.json();

    if (result.success) {

```

```

        console.log('Booking created:', result.data);
        return result.data;
    } else {
        console.error('Error:', result.error);
    }
} catch (error) {
    console.error('Network error:', error);
}
}

```

3.2 Axios Examples

```

import axios from 'axios';
// Example: Browse tours with pagination and filters \textcolor{blue}{
Updated: Now includes optional filter parameters}
async function browseTours(page = 1, size = 10, cat = null, regions =
null, priceMin = null, priceMax = null, provider = null) {
    try {
        const params = { page, size };
        if (cat && cat.length > 0) params.cat = cat.join(',');
        if (regions && regions.length > 0) params.regions = regions.join
           (',');
        if (priceMin !== null) params.priceMin = priceMin;
        if (priceMax !== null) params.priceMax = priceMax;
        if (provider) params.provider = provider;

        const { data } = await axios.get('/tours/browse', {
            params
        });

        return {
            tours: data.data.tours,
            pagination: data.data.pagination
        };
    } catch (error) {
        console.error('Error fetching tours:', error.response?.data);
        throw error;
    }
}
// Example: Submit a review
async function submitReview(reviewData) {
    try {
        const { data } = await axios.post('/reviews', {
            tour_id: reviewData.tourId,
            traveller_id: reviewData.travellerId,
            review_score: reviewData.score,
            comment: reviewData.comment
        });

        return data.data;
    } catch (error) {
        console.error('Error submitting review:', error.response?.data);
        throw error;
    }
}

```

4 Error Handling

4.1 HTTP Status Codes

- **200 OK** - Successful GET request
- **201 Created** - Successful POST request
- **400 Bad Request** - Missing required parameters or validation error
- **500 Internal Server Error** - Server-side error

4.2 Common Error Messages

- "Either agencyId or guideId is required"
- "Missing required fields: traveller_id, tour_id"
- "Type must be 'agency' or 'guide'"
- "Search term required"
- "Page >=1, size 1-100"
- "Booking rejected: Tour start date is in the past"
- "priceMin must be a non-negative number"
- "priceMax must be a non-negative number"
- "priceMin must be less than or equal to priceMax"
- "provider must be agencyör guide"

5 Notes

- All dates are in ISO 8601 format (YYYY-MM-DD)
- Timestamps include timezone information
- Rating values are cumulative (sum of all review scores)
- Average rating = `rating / num_raters`
- Tours can have either an `agency_id` OR a `guide_id`, not both
- Traveller name search is case-insensitive and supports partial matching
- `Category` and `region` filters accept comma-separated values; empty or invalid lists are ignored
- Price filters are applied as greater-than-or-equal for min and less-than-or-equal for max